

Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 4 — Operators — **ANSWERS**

*You have been using + and * since Programming 1. This is the lesson where the other three arrive — and one of them answers a question you have already asked.*

The seven arithmetic operators

OCR names seven. You already know four of them.

ERL	What it does	Python	Example
+	Addition	+	7 + 2 gives 9
-	Subtraction	-	7 - 2 gives 5
*	Multiplication	*	7 * 2 gives 14
/	Division	/	7 / 2 gives 3.5
DIV	Quotient — the whole-number part	//	7 DIV 2 gives 3
MOD	Modulo — the remainder	%	7 MOD 2 gives 1
^	Exponent — to the power of	**	2 ^ 10 gives 1024

The bottom three are the new ones. Everything below is about those.

First: division always gives you a Real

This follows straight on from data types, and it surprises people.

```
print(7 / 2)      # 3.5    -- no surprise
print(6 / 3)      # 2.0    -- ??
```

Six divided by three is two. So why does Python print 2.0?

Because / does not look at the answer before deciding what type to give you. It is the *division* operator, and division produces a **Real**. Every time. Even when the numbers divide exactly.

Now try • `print(6 / 3)` — then `print(type(6 / 3))`. What type is it?
Now try • `print(6 // 3)` — and `print(type(6 // 3))`. And that one?
Now try • From those two results, say in one sentence what // is for.

Keep the `print()` around each line. The online editor runs your code as a program, not as a calculator — a line that is only `6 / 3` works the answer out and then throws it away, and you get an empty output box and no error.

DIV and MOD — the pair that answers “how many, and what is left over”

These two always come together. Ask a primary-school division question and you get both:

7 divided by 2 is 3 remainder 1

DIV gives you the **3**. **MOD** gives you the **1**. That is the whole idea.

Worked example — 125 seconds is how many minutes and seconds?

```
total = 125
minutes = total // 60      # 2    -- how many whole minutes fit
seconds = total % 60       # 5    -- what is left over
print(str(minutes) + " min " + str(seconds) + " sec")
```

2 min 5 sec. Notice you need *both* operators to answer one question — and `str()` to join the numbers onto the text, which is casting doing its job again.

Now try • Change `total` to 3661 and run it. Is the answer sensible?

Now try • **10 days is how many weeks and days?** Write it with `//` and `%`.

Now try • `print(10 // 7)` and `print(10 % 7)` — check your answer against those.

Q1. Complete the table. Work them out first, then check at the machine.

[8]

Expression	Answer	Expression	Answer
17 DIV 5	3	17 MOD 5	2
9 DIV 3	3	9 MOD 3	0
4 DIV 9	0	4 MOD 9	4
100 DIV 7	14	100 MOD 7	2

Row 3 catches people. If the first number is smaller than the second, it fits in **zero** times and the whole thing is left over.

The trick worth knowing: MOD tests for divisibility

MOD has a second job, and it is the one exam questions use most.

If `a MOD b` is 0, then `b` divides into `a` exactly.

```
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

That is how every program you will ever write tests whether a number is even. There is no `isEven()` — there is MOD.

Q2. Write the condition that would test each of these.

[3]

(a) Is `n` divisible by 10? **`n % 10 == 0`**

(b) Is `year` a multiple of 4? **`year % 4 == 0`**

(c) Is `n` odd? **`n % 2 == 1`**

Exponent

The smallest of the three. $\wedge$ in ERL, $**$ in Python.

```
print(2 ** 10)      # 1024
print(5 ** 2)       # 25
```

Do not use $\wedge$ in Python. It compiles, it runs, and it does something else entirely (it is a bitwise operator). No error — just a wrong answer. That is a logic error, and you met those in Programming 2.

Q3. Rewrite each ERL expression in Python.

[3]

ERL	Python
$3 \wedge 4$	$3 ** 4$
total DIV count	total // count
score MOD 10	score % 10

At the machine

Both of these are on 101computing.net, where we always work.

1. Retrieval — Computing Operators Quiz. 101computing.net/computing-operators-take-the-quiz

It covers all seven arithmetic operators plus the comparison and Boolean ones. **You have not been taught AND, OR and NOT as operators yet** — if the quiz asks, have a sensible guess and we will come back to them.

2. Challenge — Time Conversion Algorithm. 101computing.net/time-conversion-algorithm

Convert a 24-hour time such as 18:36 into 12-hour format with AM or PM — 6:36 PM. The page gives you a **flowchart** and a **table of six test cases**. Use both: read the flowchart before you write anything, and do not call it finished until all six tests pass.

Where is the MOD? Twelve-hour clocks wrap around. 13 becomes 1, 14 becomes 2, 23 becomes 11. **Something MOD 12** does that wrapping for you in one line.

And the trap: what does your program print for 12:30? Work out $12 \% 12$ on paper before you run it. **It gives 0, so a naive program prints "0:30 PM" instead of "12:30 PM". The hour needs a special case. This is the single most common failure on this challenge.**

If you finish: the page has an extension task on validating the input — what should happen if somebody types 25:99?

Mark scheme

Worked solution — Time Conversion Algorithm. Run and tested, CPython 3.9.6.

```
time24 = input("Enter a 24-hour time (e.g. 18:36): ")

hours    = int(time24[0:2])      # characters 0 and 1
minutes  = time24[3:5]          # characters 3 and 4 -- left as TEXT

if hours < 12:
    period = "AM"
else:
    period = "PM"

hour12 = hours % 12              # 13 -> 1, 18 -> 6, 23 -> 11
if hour12 == 0:                 # but 0 and 12 both give 0
    hour12 = 12

print(str(hour12) + ":" + minutes + " " + period)
```

Tested: 18:36 → 6:36 PM · 00:15 → 12:15 AM · 12:30 → 12:30 PM · 09:05 → 9:05 AM · 23:59 → 11:59 PM · 12:00 → 12:00 PM.

Three things to point at when marking.

- 1. The $\% 12$ is the whole conversion** — one line does what students otherwise write eleven elifs for.
- 2. The `if hour12 == 0` is not optional.** Both midnight and midday give 0, and neither should print as "0 o'clock". This is the single most common failure.
- 3. minutes is never converted to a number.** It is only ever printed, so leaving it as text keeps 09:05 printing as 9:05 rather than 9:5. A student who casts it to int loses the leading zero — and there it is again.

*Total: 14 marks on the written questions. **Next lesson: string manipulation** — and you will need DIV and MOD there too.*